LAB 12: ALGORITHMS WITH AI ASSISTANCE

Sorting, Searching, and Optimizing Algorithms

Name: Roger A Raju

Rollno: 2503a52370

batch-13

Task 1: Merge Sort Implementation

Question:

Write merge_sort(arr) to sort ascending, include time and space complexity in the docstring, and verify with test cases.

Prompt:

Generate a documented merge_sort(arr) with a merge helper. Include complexity and test empty, single-item, duplicate, and unsorted lists.

Python Code:

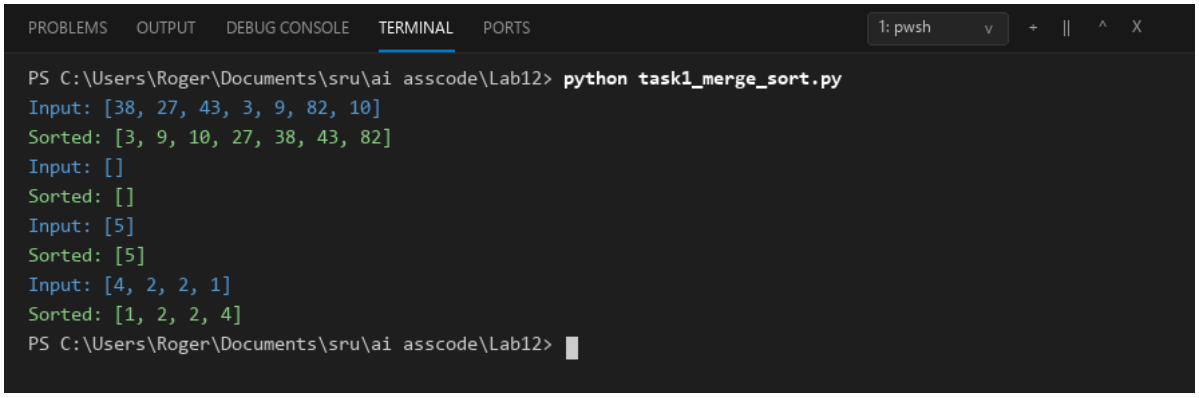
```
def merge_sort(arr):
    """Sort ascending. Time: O(n log n) in all cases; extra space: O(n)."""
    if len(arr) <= 1:
        return arr[:]
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)


def merge(left, right):
    """Merge two sorted lists."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result


if __name__ == "__main__":
    for values in ([38, 27, 43, 3, 9, 82, 10], [], [5], [4, 2, 2, 1]):
```

```
        print("Input:", values)
        print("Sorted:", merge_sort(values))
```

Output :



Justification:

Merge Sort uses divide-and-conquer. Time complexity is O(n log n) in best, average, and worst cases; auxiliary space is O(n). Tests include empty and duplicate values.
Run in VS Code terminal: python task1_merge_sort.py

Task 2: Binary Search with AI Optimization

Question:

Create binary_search(arr, target), returning the index or -1. Explain best, average, and worst-case complexities and test different inputs.

Prompt:

Implement iterative binary search on an ascending sorted list, return index or -1, document complexities, and test found and missing targets.

Python Code:

```
def binary_search(arr, target):
    """Search an ascending sorted list; return index or -1.
    Best time: O(1); average/worst time: O(log n); space: O(1).
    The input must already be sorted.
    """
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

```
if __name__ == "__main__":
    numbers = [3, 8, 12, 17, 25, 31, 42]
    for target in [17, 3, 42, 99]:
        print(f"Target {target}: index {binary_search(numbers, target)}")
```

Output :



Justification:

Binary Search halves the search interval each step. Best-case time is O(1), average/worst-case time is O(log n), and iterative extra space is O(1). Input must be sorted.
Run in VS Code terminal: python task2_binary_search.py

Task 3: Real-Time Application Challenge – Inventory Management System

Question:

Search thousands of products by ID/name and sort by price/stock quantity. Recommend and justify algorithms considering dataset size, update frequency, and performance.

Prompt:

Implement product ID lookup, case-insensitive name search, and sorting by price/quantity. Add sample data, docstrings, complexity notes, and an algorithm selection table.

Code:

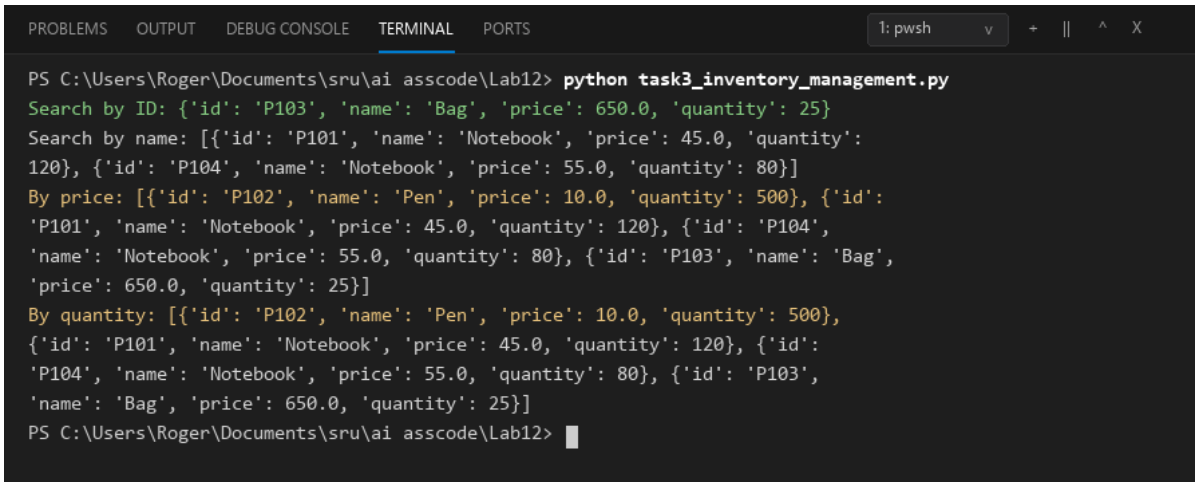
```
def search_by_id(products, product_id):
    """Average O(1) dictionary lookup."""
    return products.get(product_id)


def search_by_name(products, name):
    """Case-insensitive exact-name search; O(n)."""
    target = name.strip().casefold()
    return [p for p in products.values() if p["name"].casefold() == target]


def sort_inventory(products, field="price", descending=False):
    """Sort by price or quantity in O(n log n)."""
    if field not in {"price", "quantity"}:
        raise ValueError("field must be 'price' or 'quantity'")
    return sorted(products.values(), key=lambda p: p[field], reverse=descending)
```

```
if __name__ == "__main__":
    inventory = {
        "P101": {"id": "P101", "name": "Notebook", "price": 45.0, "quantity": 120},
        "P102": {"id": "P102", "name": "Pen", "price": 10.0, "quantity": 500},
        "P103": {"id": "P103", "name": "Bag", "price": 650.0, "quantity": 25},
        "P104": {"id": "P104", "name": "Notebook", "price": 55.0, "quantity": 80}
    }
    print("Search by ID:", search_by_id(inventory, "P103"))
    print("Search by name:", search_by_name(inventory, "Notebook"))
    print("By price:", sort_inventory(inventory, "price"))
    print("By quantity:", sort_inventory(inventory, "quantity", True))
```

Output :



Justification:

Dictionary ID lookup is average O(1); name scanning is O(n), with a secondary index useful for frequent name searches. Python sorted() uses Timsort and is O(n log n). Keep indexes current as products change.

Run in VS Code terminal: python task3_inventory_management.py